DESIGN AND ANALYSIS OF ALGORITHMS

COMPETITIVE PROGRAMMING

1. Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

ANSWER:

```
#include <stdio.h>
int main() {
   int n;
   scanf("%d", &n);
   int a[n];
   for (int i = 0; i < n; i++) {
      scanf("%d", &a[i]);
   }
```

```
    // Compare every element with the elements after it
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

2. Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

ANSWER:

```
#include <stdio.h>
int main() {
   int n;
   scanf("%d", &n);
   int a[n];
   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   int slow = a[0];
   int fast = a[0];
   do {
      slow = a[slow];
      fast = a[a[fast]];
   } while (slow != fast);
   slow = a[0];
   while (slow != fast) {
      slow = a[slow];
      fast = a[fast];
   }
   printf("%d", slow);
   return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

3. Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

- **The first line contains T, the number of test cases. Following T lines contain:**

1. **Line 1 contains N1, followed by N1 integers of the first array**
2. **Line 2 contains N2, followed by N2 integers of the second array**

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

ANSWER:

```
#include <stdio.h>
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        int m, n;
        scanf("%d", &m);
        int a[m];
        for (int i = 0; i < m; i++)
            scanf("%d", &a[i]);
        scanf("%d", &n);
        int b[n];
        for (int i = 0; i < n; i++)
            scanf("%d", &b[i]);
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (a[i] == b[j]) {
                    printf("%d ", a[i]);
                    break;
                }
            }
        }
```

```
        printf("\n");
    }
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

4. Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

- **The first line contains T, the number of test cases. Following T lines contain:**

1. **Line 1 contains N1, followed by N1 integers of the first array**
2. **Line 2 contains N2, followed by N2 integers of the second array**

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

ANSWER:

```
#include <stdio.h>
int main() {
   int T;
   scanf("%d", &T);
   while (T--) {
      int m, n;
      scanf("%d", &m);
      int a[m];
      for (int i = 0; i < m; i++)
         scanf("%d", &a[i]);
      scanf("%d", &n);
      int b[n];
      for (int i = 0; i < n; i++)
         scanf("%d", &b[i]);
      int i = 0, j = 0;
      while (i < m && j < n) {
```

```
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
            }
        }
        printf("\n");
    }
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

5. Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

ANSWER:

```
#include <stdio.h>
int main() {
   int n, k;
   scanf("%d", &n);
   int a[n];
   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   scanf("%d", &k);
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (i != j && a[j] - a[i] == k) {
            printf("1");
            return 0;
         }
      }
   }
   printf("0");
```

```
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.

6. Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

ANSWER:

```
#include <stdio.h>
int main() {
    int n, k;
```

```
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &k);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && a[j] - a[i] == k) {
                printf("1");
                return 0;
            }
        }
    }
    printf("0");
    return 0;
}
```

RESULT:

Thus the code is verified and the expected output is obtained.